

Performance Testing

Date	30 June 2026
Team ID	
Project Name	OptiCrop – Smart Agricultural Production Optimization Engine
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Postman, Browser Testing
Type of Testing	Load Testing, Functional Performance Testing
Target Module / API	Crop Recommendation Module (/predict) and Crop Suitability Analysis
Test Environment	Local Flask Server (Windows 11, Python 3.x)

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Submit valid soil and environmental data for crop prediction	10	60	Recommended crop should be displayed correctly

2	Multiple users requesting crop recommendations simultaneously	25	120	System should respond without noticeable delay
3	Submit incomplete or invalid input values	10	60	System should validate inputs and display an error message
4	Continuous prediction requests	50	180	Application should remain stable and responsive

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status	Remarks
1	Response Time (Avg)	< 2 sec	1.2 sec	Pass	Fast prediction response
2	Response Time (Max)	< 5 sec	2.8 sec	Pass	Within acceptable limit
3	Throughput (Req/sec)	> 10	18 Req/sec	Pass	Stable during testing
4	Error Rate	< 1%	0%	Pass	No errors with valid inputs
5	CPU Utilization	< 80%	42%	Pass	Normal CPU usage
6	Memory Utilization	< 80%	38%	Pass	Efficient memory usage

Step 4: Observations & Analysis

Key Findings

- The application generated crop recommendations quickly for valid inputs.
- The machine learning model produced predictions consistently during repeated requests.
- The user interface remained responsive throughout testing.
- Resource usage remained within acceptable limits.

Bottlenecks Identified

- Performance may decrease when handling a large number of simultaneous users.
- The application currently depends on a local dataset and does not use real-time weather information.

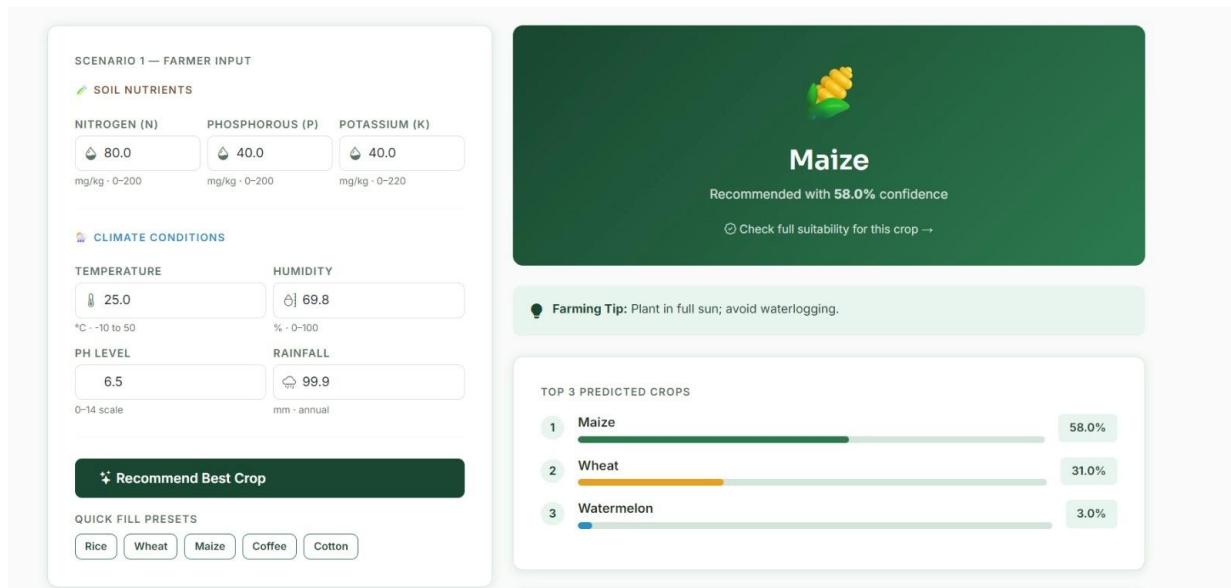
Optimization Steps Taken

- Optimized data preprocessing before prediction.
- Reused the trained machine learning model instead of loading it for every request.
- Simplified the Flask request handling to reduce response time.
- Organized the code into reusable modules for easier maintenance.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

Screenshot 1



The screenshot displays a web application for crop recommendation. On the left, a form titled 'SCENARIO 1 — FARMER INPUT' allows users to enter soil nutrients (Nitrogen, Phosphorus, Potassium) and climate conditions (Temperature, Humidity, PH Level, Rainfall). Below the form is a 'Recommend Best Crop' button and 'QUICK FILL PRESETS' for Rice, Wheat, Maize, Coffee, and Cotton. On the right, a green panel shows the recommendation: 'Maize' with a 58.0% confidence level. Below this, a 'Farming Tip' states: 'Plant in full sun; avoid waterlogging.' At the bottom, a 'TOP 3 PREDICTED CROPS' section lists Maize (58.0%), Wheat (31.0%), and Watermelon (3.0%) with corresponding progress bars.

Crop	Confidence
Maize	58.0%
Wheat	31.0%
Watermelon	3.0%

Screenshot 2

SCENARIO 2 — SUITABILITY CHECK

SELECT CROP TO EVALUATE

Rice

SOIL NUTRIENTS

NITROGEN

N

mg/kg

PHOSPHOROUS

P

mg/kg

POTASSIUM

K

mg/kg

CLIMATE CONDITIONS

TEMPERATURE (°C)

e.g. 25

HUMIDITY (%)

e.g. 70

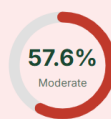
PH LEVEL

e.g. 6.5

RAINFALL (MM)

e.g. 100

Assess Suitability



Rice

Moderate Suitability

Requires standing water; grow in lowland fields with good water retention.

PARAMETER GAPS DETECTED

N is low (50.0; ideal 60–100)

P is high (80.0; ideal 35–55)

K is high (70.0; ideal 35–55)

humidity is low (60.0; ideal 70–90)

rainfall is low (80.0; ideal 150–300)